Sampled edges, serial complement and RTL questions

Keep the recovered questions, detailed Moore complementer, small RTL mistakes and dated submission evidence together.

Kapil Tripathi | Verilog and digital design | 24 pages

CONTENTS / click a topic to jump

Original entry 80 request: "explain me this code in a deep manner, along with need of each state"; "ik the meaning and need of states s0, s1, s2 ... rest, explain me in doc".

Reading days group the explanations, including later revisits. Tracker day labels and original dated submission evidence remain unchanged.

Use the linked contents or PDF bookmarks. Page numbers match the PDF viewer. Source questions and technical evidence remain beside their explanations.

Questions answered

A synchronous edge is a difference between samples

The circuit observes one vector value at each rising clock edge and compares it with the value stored from the preceding edge.

Original submitted-code question

```
// is it like 0 to 1 transition for any clock edge ? like in any 2 clock edges its that, transition
```

Answer

Yes: for each bit, a positive edge is detected when the preceding rising-edge sample was 0 and the current rising-edge sample is 1. It is not a detector for every physical transition between clocks. A short pulse that rises and falls between two sampling edges can be missed.

Previous sample	Current sample	in & ~previous	Meaning
0	0	0	no transition
0	1	1	sampled positive edge
1	0	0	falling edge, not requested
1	1	0	still high

Correct sequential implementation

```
reg [7:0] previous;

always @(posedge clk) begin
    previous <= in;
    pedge    <= in & ~previous;
end
```

Why both assignments work in one block

Nonblocking assignments evaluate their right-hand sides using the values that existed when the block began. Therefore **pedge** uses the old **previous**, while **previous <= in** schedules the current sample for later. After the time step, the history register advances and the one-cycle pulse is available.

Clock	old previous	sampled in	pedge computed	new previous
n-1	0	0	0	0
n	0	1	1	1
n+1	1	1	0	1

Observed mistake

The unsuccessful expression **~in & previous** selects previous=1 and current=0, so it detects a sampled 1 -> 0 falling edge instead.

An asynchronous reset must wake the block

This active-low reset must clear the state immediately when it falls, even if no rising clock edge occurs.

Original submitted-code question

```
// why i have to write, asynchronous reset in the sensitivity list
```

Answer

An always block runs only when an event in its event control occurs. If reset is asynchronous, reset assertion itself must be one of those events. The signal is named **aresetn**: the suffix **n** means active low, so the assertion event is **negedge aresetn**.

```
always @(posedge clk or negedge aresetn) begin
    if (!aresetn)
        state <= S_NONE;
    else
        state <= next_state;
end
```

Form	When reset changes state	Hardware described
@(posedge clk)	Only at a rising clock edge	Synchronous reset if reset is checked inside
@(posedge clk or negedge aresetn)	Immediately when aresetn falls	Active-low asynchronous-reset flip-flops
@(posedge clk or posedge areset)	Immediately when areset rises	Active-high asynchronous-reset flip-flops

Event-by-event behavior

Event	aresetn	What the block does
negedge aresetn	becomes 0	Run immediately and load S_NONE
posedge clk while reset=0	still 0	Run and keep S_NONE
posedge aresetn	becomes 1	No state update; wait for the next clock
next posedge clk	1	Load next_state normally

Why not write bare aresetn?

The design needs a specific assertion edge that maps to a resettable flip-flop. **negedge** states both the polarity and the hardware event; omitting reset from the event control changes the behavior to synchronous.

Physical-design note: asynchronous assertion is often combined with synchronized deassertion to avoid recovery/removal timing problems. HDLBits is testing the RTL reset behavior shown above.

Reduction, bitwise, and logical operators

The fastest way to choose correctly is to ask whether the desired result is one bit, one bit per vector position, or one Boolean truth result per whole operand.

Original submitted-code question

```
// tell me difference b/w clearly b/w reduction and logical and bitwise
```

Class	Typical form	Operation	Result width
Reduction	&in, |in, ^in	Combine every bit of one vector	1 bit
Bitwise	a & b, a | b, a ^ b	Combine aligned bit positions independently	Vector width
Logical	a && b, a || b, !a	Treat each whole operand as zero/nonzero	1 bit

Worked example

Let **a = 4'b1010** and **b = 4'b1100**.

Expression	Result	Reason
&a	1'b0	not every bit of a is 1
|a	1'b1	at least one bit of a is 1
^a	1'b0	a contains an even number of 1 bits
a & b	4'b1000	AND each aligned pair of bits
a && b	1'b1	both complete vectors are nonzero
!a	1'b0	a is not the all-zero vector

Correct answer for the four-input-gates problem

```
assign out_and = &in;
assign out_or  = |in;
assign out_xor = ^in;
```

Why the submitted compound forms were wrong

An expression such as **out_and &= in** is a compound read-modify-write assignment, conceptually **out_and = out_and & in**. It reads the previous output and does not reduce the bits of **in**. In combinational logic it also creates a self-dependency that can imply unwanted feedback.

One-line selection rule

One vector in and one bit out: reduction. Two vectors and a vector out: bitwise. Whole conditions in if/while logic: logical.

Revision checklist and verification record

Use this page as the quick recall sheet before resubmitting related HDLBits problems.

Entry	Recall question	Correct recall
54	Can inputs replace states?	No. Inputs are present observations; states preserve required history.
74	What is a sampled positive edge?	previous=0 and current=1 at consecutive rising-edge samples.
75	Why is reset in the event control?	An asynchronous assertion must run the block without waiting for clk.
77	Which operator collapses a vector?	A unary reduction operator such as &in, |in, or ^in.

Verification performed

- The original HDLBits pages and saved submitted-code comments were inspected in the signed-in Chrome session on 2026-09-02.
- The reusable Icarus Verilog self-check passed the corrected behavior for entries 74, 75, and 77.
- Entry 54 is a conceptual state-model answer; the generic snippet illustrates the separation of input decoding, state encoding, and remembered history rather than claiming a full replacement submission.
- The tracker links each question to its page in Day 3 or Day 5.

Questions covered

Water-level history: Day 3, page 3. The other three answers: pages 3-5 here. The water-level snippet is not a full solution; the saved Day 5 testbench checks the other examples.

Original HDLBits references

Entry 54 - exams/ece241_2013_q4

Entry 74 - edgedetect

Entry 75 - exams/ece241_2013_q8

Entry 77 - gates4

Submission reminder

A locally correct explanation or self-check is supporting evidence. HDLBits itself remains the authority for whether a submitted solution is accepted.

1. What the serial machine computes

For a fixed-width binary word, two's complement means invert all bits and add one. That textbook operation looks global, but an LSB-first stream exposes a simpler local rule. The carry from the added one moves from low bits toward high bits, exactly in the same direction as the incoming stream.

Streaming rule

While the input is `0`, adding one keeps propagating, so output `0`. The first input `1` absorbs the carry and is copied as output `1`. Once that first `1` has appeared, the carry is finished, so every remaining input bit is inverted.

Why the rule is equivalent to invert-plus-one

- All trailing zeros below the least-significant `1` remain zero in the two's complement.
- The least-significant `1` itself remains one; it is the bit that terminates the add-one carry.
- Every more-significant bit above it is complemented.
- If the entire stream is zero, no first one ever occurs, and the output correctly remains all zero.

Worked 8-bit example

Take the parallel value `8'b0010_1100` (`8'h2C`). Its two's complement is `8'b1101_0100` (`8'hD4`). The serial order below is bit 0 first, so read each row from left to right in time.

Clock sample	0	1	2	3	4	5	6	7
Input x	0	0	1	1	0	1	0	0
Action	copy	copy	copy first 1	invert	invert	invert	invert	invert
Output z	0	0	1	0	1	0	1	1

Important direction detail

The method works online only because the least-significant bit arrives first. With an MSB-first stream, the circuit would not yet know where the least-significant one is, so it could not decide immediately which early bits to copy or invert.

2. Why a Moore machine needs three states

A state is not merely a label for the last input. It represents every piece of remembered history that can change present output or future behavior. Because this is a Moore machine, the output is a function of state alone: the input may choose the next state, but it cannot directly appear in the output equation.

Code state	Semantic name	What history it represents	Moore z
S0	COPY_ZERO	No 1 has been seen yet. The add-one carry is still conceptually propagating.	0
S1	OUTPUT_1	The bit just sampled must produce 1: either it was the first 1, or it was a later 0 being inverted.	1
S2	OUTPUT_0	A first 1 was already seen, and the bit just sampled was a later 1 being inverted.	0

Why S0 and S2 cannot be one state

Both states output `0`, but equal output does not make two states equivalent. Give both states the same next input `x=0`: from `S0`, that zero is still before the first one, so it must be copied and the next output stays `0`; from `S2`, the first one was already seen, so that zero must be inverted and the next output becomes `1`. Because the same future input requires different future behavior, the two histories must remain distinct.

Minimality argument

A Moore state partition must separate: (1) pre-first-one with output `0`, (2) post-first-one with output `1`, and (3) post-first-one with output `0`. Those are three distinguishable classes, so three Moore states are sufficient and, for this interface, necessary.

Why there is no S3

Two state bits provide four encodings, but the behavior needs only three states. `2'b11` is therefore unused. The `default` case sends any illegal or unknown encoding back to `S0`. The unused encoding is capacity, not evidence that a fourth behavioral state is missing.

A better way to name the states

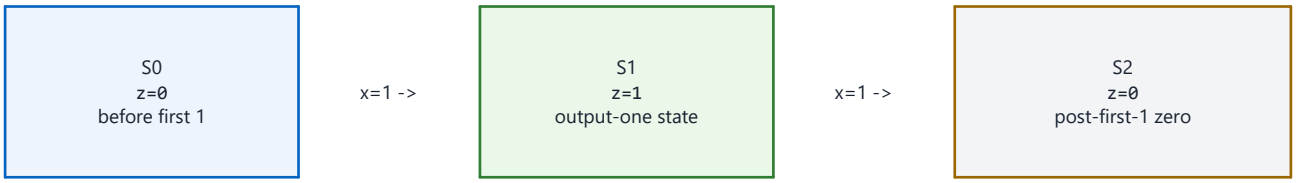
Names such as `COPY_ZERO`, `OUTPUT_1`, and `OUTPUT_0` expose the invariant directly. Numeric names are legal, but semantic names prevent the common mistake of treating `S1` as simply 'input was 1'. In fact, `S1` can be entered after a post-first-one input of zero because that zero must be inverted to one.

3. Every transition, decoded

The table should be read at a rising edge: start in the current state, sample x, choose next_state, update state, and then observe z from the new current state.

Current	x	Next	z after edge	Reason
S0	0	S0	0	Still before the first 1; copy the zero.
S0	1	S1	1	This is the first 1; copy it and enter inversion phase.
S1	0	S1	1	Already past the first 1; invert 0 to 1.
S1	1	S2	0	Already past the first 1; invert 1 to 0.
S2	0	S1	1	Already past the first 1; invert 0 to 1.
S2	1	S2	0	Already past the first 1; invert 1 to 0.

The complete state picture



Self-loops: `S0 --x=0--> S0`, `S1 --x=0--> S1`, and `S2 --x=1--> S2`. The cross-edge `S2 --x=0--> S1` completes the inversion rule.

The line that fixed the unsuccessful version

The successful code uses `S2: next_state = x ? S2 : S1;`. In the earlier non-success, the two destinations were swapped. After the first one, input `1` must produce output `0` and therefore go to `S2`; input `0` must produce output `1` and therefore go to `S1`.

4. Clock-by-clock timing trace

The example uses input `8'b0010_1100` in parallel notation, supplied serially as bit 0 through bit 7. Reset first places the machine in `S0`. Each row shows the state after that row's rising edge, which is also when the corresponding output bit becomes valid.

Edge	Sampled x	State before	State after	z after	Interpretation
reset	-	unknown	S0	0	Asynchronous reset establishes copy phase.
1	0	S0	S0	0	Copy a zero before the first one.
2	0	S0	S0	0	Copy another zero.
3	1	S0	S1	1	Copy the first one; inversion phase begins.
4	1	S1	S2	0	Invert later one to zero.
5	0	S2	S1	1	Invert later zero to one.
6	1	S1	S2	0	Invert later one to zero.
7	0	S2	S1	1	Invert later zero to one.
8	0	S1	S1	1	Invert later zero to one.

Is this Moore output one cycle late?

A Moore output is determined by the state that exists at the instant you observe it. Immediately before a rising edge, `z` still describes the previous state. At the edge, `x` is sampled and `state <= next_state` schedules the state update. After the nonblocking-assignment update, the continuous assignment recomputes `z` from the new state. HDLBits compares the output in this post-edge sense, so the output bit associated with the sampled input is available just after that edge.

> **Do not sample in the wrong phase**
>
> If a testbench checks `z` in the same simulation region as `@(posedge clk)` without allowing the nonblocking assignment to settle, it may see the previous value and falsely report a one-cycle error. Check after a small delay or in a later clocking region.

What asynchronous reset changes

Because the register block is sensitive to both `posedge clk` and `posedge areset`, asserting reset changes state to `S0` without waiting for a clock. Since `z` is decoded from state, it then returns to zero as well. Releasing reset does not itself trigger the block; normal conversion resumes on the next rising clock edge.

5. The successful code, block by block

```
module top_module (
    input clk, input areset, input x, output z
);
    reg [1:0] state, next_state;
    parameter S0 = 2'b00;
    parameter S1 = 2'b01;
    parameter S2 = 2'b10;

    always @(posedge clk or posedge areset) begin
        if (areset)
            state <= S0;
        else
            state <= next_state;
    end

    always @(*) begin
        next_state = state;
        case (state)
            S0: next_state = x ? S1 : S0;
            S1: next_state = x ? S2 : S1;
            S2: next_state = x ? S2 : S1;
            default: next_state = S0;
        endcase
    end

    assign z = (state == S1);
endmodule
```

Declarations and encoding

- `state` stores the current Moore state. `next_state` is purely combinational and becomes the next registered value.
- Three states require at least two state bits because `ceil(log2(3)) = 2`. The fourth binary code is unused.

State register

- The event control implements a positive-edge clock and positive-level asynchronous reset assertion: `@(posedge clk or posedge areset)`.
- Nonblocking assignment models simultaneous flip-flop updates and avoids procedural ordering artifacts.

Next-state block

- `always @(*)` automatically includes every read signal in the sensitivity list for combinational logic.
- `next_state = state` gives a safe hold default. Every legal state is then explicitly decoded, and `default` recovers from an unused encoding.

Output decode

- `assign z = (state == S1)` depends only on state, which is the defining Moore property. It is one only in the state whose semantic meaning is 'the current output bit must be one'.

6. Cleaner names and the Mealy contrast

Here is the same SystemVerilog design with names that describe each state:

```
module top_module (
    input  logic clk,
    input  logic areset,
    input  logic x,
    output logic z
);
    typedef enum logic [1:0] {
        COPY_ZERO = 2'b00,
        OUTPUT_1  = 2'b01,
        OUTPUT_0  = 2'b10
    } state_t;

    state_t state, next_state;

    always_ff @(posedge clk or posedge areset)
        if (areset) state <= COPY_ZERO;
        else        state <= next_state;

    always_comb begin
        case (state)
            COPY_ZERO: next_state = x ? OUTPUT_1 : COPY_ZERO;
            OUTPUT_1:  next_state = x ? OUTPUT_0 : OUTPUT_1;
            OUTPUT_0:  next_state = x ? OUTPUT_0 : OUTPUT_1;
            default:   next_state = COPY_ZERO;
        endcase
    end

    assign z = (state == OUTPUT_1);
endmodule
```

Why a Mealy solution needs only two states

A Mealy output may depend on both state and current input. It needs only a phase bit: `COPY` means no first one has appeared, with z=x; `INVERT` means the first one has appeared, with z=~x. The Moore version cannot put x directly in the output expression, so it must split the post-edge result into OUTPUT_1 and OUTPUT_0 states and also keep COPY_ZERO separate from the post-first-one zero state.

Architecture	Remembered states	Output rule	Key tradeoff
Moore	3	z = function(state)	More states; output changes only with state.
Mealy	2	z = function(state, x)	Fewer states; output has a direct combinational input path.

Do not mix the two designs accidentally

Your accepted entry 80 is Moore and your accepted entry 85 is Mealy. Both implement the same serial arithmetic rule, but their state meanings and output timing equations are intentionally different.

7. Mistakes, verification, and recall

Mistakes worth remembering

Mistake	Why it fails	Correct recall
Swap S2 destinations	It maps post-first-one input 1 to output 1 and input 0 to output 0.	In inversion phase: 1 -> S2/z0, 0 -> S1/z1.
Merge S0 and S2	They both output zero but react differently to a future zero.	Same output does not imply equivalent future behavior.
Treat S1 as last input was 1	S1 is also reached when a later input zero is inverted.	Name states by phase and required output, not by raw input.
Read z before NBA update	The testbench observes the previous state value.	Associate each sampled bit with z just after the corresponding edge.
Add an S3 because 2 bits allow it	Encoding capacity is not a behavioral requirement.	Three distinguishable Moore situations need three states.

Verification record

- The signed-in Chrome submission selector reports a fresh successful submission for entry 80 at 9/2/2026, 4:28:11 PM.
- The latest non-success at 9/2/2026, 4:18:12 PM used the incorrect swapped S2 transition; the successful version corrects it.
- The reusable Icarus Verilog self-check passed the Moore two's-complement behavior together with the other reviewed entries.
- The trace in this document reconstructs 8'h2C -> 8'hD4 in LSB-first order.

Five-question recall test

Recall question	Answer
What happens before the first 1?	Copy input bits unchanged.
What happens after the first 1?	Invert every later input bit.
Why are S0 and S2 separate?	They output 0 but have different future behavior for x=0.
Which state produces z=1?	Only S1 / OUTPUT_1.
What is the correct S2 transition?	x=1 stays S2; x=0 goes to S1.

Why the states are needed

S0 exists to remember that the first one has not appeared. S1 exists because a Moore machine needs a state whose output is one. S2 exists because, after the first one, output zero has different future behavior from the pre-first-one zero of S0. The transition table is exactly the serial two's-complement algorithm expressed as Moore state memory.

Original HDLBits entry 80 page: exams/ece241_2014_q5a

Day 5 questions and submission notes

This review explains the saved code for entries 81-85 and keeps the earlier questions for entries 54, 74, 75, 77 and 80 nearby. The submission records at the end were collected on 2-3 September 2026. They describe that checkpoint; the tracker holds the current progress.

Entries 81-85 are all Done in the tracker. The main fixes are Boolean OR, the two padding bits in a concatenation, sticky edge capture, the ring/vibrate condition, and one-hot Mealy state encoding. Entry 80 has its own detailed Moore explanation.

This chapter preserves the review and its historical submission table. The Markdown companion remains available from the collection index; tracker links open the combined day-wise PDFs.

The saved records show success through entry 93 at the original checkpoint. Entries 86-93 were confirmed by Kapil on 3 September. A matching tab alone does not show that its current editor contents pass; later experiments can differ from an earlier successful submission.

Earlier questions and where to read them

Entry	Exact subject found in submitted code	Existing answer
7	Why the Lemmings4 fall counter does not need another reset	Combined questions PDF, page 8
13	Why bitwise operators are used in the vector mux	Combined questions PDF, section 6
26	Why `initial` cannot initialize a live population count and when to use procedural versus generate loops	Combined questions PDF, sections 2, 3, and 5
29	How `sel[0]` and `sel[1]` divide work between the two mux levels	Combined questions PDF, section 6
30	Whether the 101-node carry chain should be a wire or register	Combined questions PDF, section 4
31	Difference between bitwise `~` and logical `!`	Combined questions PDF, section 9
36	Why `&` and `&&` happen to choose the same branch with an all-ones mask	Combined questions PDF, section 10
38	What latch inference means for an intentionally incomplete assignment	Combined questions PDF, section 11
67	Meaning and tokenization of indexed part-select operators `+:` and `-:`	Day 4 questions PDF, pages 2-3
70	Why the PS/2 data register must not be cleared in an unmatched clocked `else` branch	Day 4 questions PDF, PS/2 datapath section

Question 1 - Entry 54, water-level Moore FSM

Original code comment:

```
//localparam s2 // i dont need the check for specific cases cause they are already in the input
```

The input tells the controller which water-level sensors are currently asserted, but it does not necessarily contain all the history needed by the outputs. In this problem, the ordinary flow outputs are determined by

the current level band, while the supplemental-flow decision also distinguishes whether the level arrived from above or below. Two moments can therefore have the same current sensor pattern but require different `dfr` behavior because their preceding level was different.

`localparam` declarations do not test inputs. They give readable binary encodings to internal states. States such as `belows2` and `aboves2` preserve the direction/history that the input vector alone cannot express. The combinational next-state block must still test the valid sensor bands to decide which history state comes next.

If the HDLBits statement guarantees physically valid sensor patterns, it is reasonable to prioritize the highest asserted sensor rather than enumerate impossible patterns separately. That does not remove the need for the history states. This fragment shows the idea (it is not a complete submission):

```
always @(*) begin
    next_state = state;
    if (s[3])
        next_state = ABOVE_S3;
    else if (s[2])
        next_state = (state_was_higher) ? BELOW_S3 : ABOVE_S2;
    else if (s[1])
        next_state = (state_was_higher) ? BELOW_S2 : ABOVE_S1;
    else
        next_state = BELOW_S1;
end
```

The exact state names may differ, but the principle is fixed: sensor bits describe the present level; state bits preserve any past information the outputs still need.

Question 2 - Entry 74, sampled positive-edge detection

Original code comment:

```
// is it like 0 to 1 transition for any clock edge ? like in any 2 clock edges its that, transition
```

Yes, but "edge" here means a change between **two consecutive rising-clock samples**, not every physical transition that might occur between clocks. For each vector bit:

- `previous[i] = 0` at the preceding rising edge,

- `in[i] = 1` at the current rising edge,

- therefore `in[i] & ~previous[i] = 1` for one cycle.

The sequential implementation is:

```
reg [7:0] previous;

always @(posedge clk) begin
    previous <= in;
    pedge    <= in & ~previous;
end
```

Both right-hand sides see the old `previous` value because nonblocking assignments update after the block has evaluated. The result is one pulse per sampled 0-to-1 transition. A pulse that rises and falls entirely between two clock edges may be missed because this is synchronous sampling, not an asynchronous edge detector.

The unsuccessful submission used `~in & previous`, which detects the opposite sampled transition: 1-to-0.

Question 3 - Entry 75, asynchronous-reset sensitivity list

Original code comment:

```
// why i have to write, asynchronous reset in the sensitivity list
```

An asynchronous reset must change the state register immediately when reset is asserted, without waiting for a clock edge. The event control must therefore wake the block for either event:

```
always @(posedge clk or negedge aresetn) begin
    if (!aresetn)
        state <= S0;
    else
        state <= next_state;
end
```

`aresetn` is active low, so assertion is its falling edge, written `negedge aresetn`. A bare `aresetn` in the event list does not describe the required flip-flop primitive and does not state which transition asserts reset. Omitting reset from the list would make the reset synchronous because the block could react only at `posedge clk`.

In physical designs, asynchronous assertion is often paired with synchronized deassertion to avoid recovery/removal timing problems, but the HDLBits problem specifically tests the basic active-low asynchronous-reset behavior.

Question 4 - Entry 77, reduction versus bitwise versus logical operators

Original code comment:

```
// tell me difference b/w clearly b/w reduction and logical and bitwise
```

Operator class	Example	Input/output width	Meaning for `in = 4'b1011`
Unary reduction	`&in`, `|in`, `^in`	Vector to one bit	`&in=0`, `|in=1`, `^in=1`
Bitwise binary	`a & b`, `a | b`, `a ^ b`	One result bit per aligned input bit	Combines corresponding bits independently
Logical	`a && b`, `a || b`, `!a`	Operands become true/false; result is one bit	Tests whether each whole operand is zero or nonzero

The original problem asks for one output from all four bits, so unary reduction operators are the direct answer:

```
assign out_and = &in;
assign out_or  = |in;
assign out_xor = ^in;
```

The unsuccessful forms `out_and &= in`, `out_or |= in`, and `out_xor ^= in` are compound read-modify-write assignments. They read the old output and combine it with `in`; they are not reduction operators. In combinational logic, reading an output while assigning that same output can also create an unintended feedback dependency.

Submission notes for entries 80-85

Entry	Chrome evidence	Attempt 2 state	Finding
80	Last success 2026-09-02 16:28:11; last non-success 16:18:12	Done	The corrected Moore implementation passed, and its new in-code explanation request is answered in a dedicated PDF.
81	Last success 2026-09-01 23:26:08	Done	Waveform implements `q = b | c`; inputs `a` and `d` are irrelevant.
82	Last success 2026-09-01 23:23:20	Done	Correct 32-bit concatenation includes two trailing one bits.
83	Last success 2026-09-02 17:00:08; last non-success 16:57:13	Done	The successful version keeps captured falling edges sticky and initializes the previous sample during reset.
84	Last success 2026-09-02 16:54:21; last non-success 16:53:47	Done	The successful motor equation now requires both `ring` and `vibrate_mode`.
85	Last success 2026-09-02 16:44:54; last non-success 16:31:02	Done	The two-state Mealy behavior passed; the internal encoding is still not literally one-hot.

Entry 80 - Q5a serial two's complementer, Moore FSM

For an LSB-first stream, two's complement can be produced by copying zeros until the first 1, copying that first 1, and inverting every later bit. A Moore output cannot depend directly on the current input, so the output value must be represented by the state reached after sampling that input. This needs three functional states:

- `COPY_ZERO`, output 0, before the first 1;

- `OUTPUT_1`, output 1, for the first 1 or a later inverted 0;

- `OUTPUT_0`, output 0, for a later inverted 1.

The saved successful code follows this rule and includes a `default` transition for the unused state encoding. Its submitted comments explicitly ask for a deeper explanation of the algorithm, timing, and need for each state; see the Entry 80 Moore FSM deep-dive PDF.

Entry 81 - Combinational circuit 4

The waveform shows that `q` depends only on `b` and `c`:

```
assign q = b | c;
```

The old `b + c` expression is arithmetic addition, not Boolean OR. When `b=c=1`, the mathematical sum is binary 2 and the carry cannot be represented by the one-bit output, while OR must remain 1. This counterexample distinguishes the two immediately.

Entry 82 - Vector concatenation

Six five-bit inputs contain 30 bits, while `{w,x,y,z}` contains 32 bits. The original prompt explicitly requires two one bits after the six vectors:

```
assign {w, x, y, z} = {a, b, c, d, e, f, 2'b11};
```

Omitting the final two bits does not merely leave `z[1:0]` unspecified. The 30-bit value is extended to the 32-bit destination from the left, so the grouping of all output bytes is shifted relative to the required concatenation.

Entry 83 - Sticky falling-edge capture

The latest non-success overwrote `solution` with only the newest falling-edge mask. The prompt requires every detected bit to remain set until reset, so the new mask must be ORed with the old output. The previous input must also be sampled on every edge, including reset edges, so the first comparison after reset has a defined predecessor. The latest successful submission applies both corrections.

```
reg [31:0] previous;

always @(posedge clk) begin
    previous <= in;
    if (reset)
        out <= 32'b0;
    else
        out <= out | (previous & ~in);
end
```

Reset has priority over capture. `previous & ~in` detects bits that were 1 at the preceding sample and are 0 now. The OR makes the result sticky.

Entry 84 - Ring or vibrate

The required truth table is:

ring	vibrate_mode	ringer	motor
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

Therefore:

```
assign ringer = ring & ~vibrate_mode;
assign motor  = ring &  vibrate_mode;
```

The unsuccessful equation used `vibrate_mode & !ring`, which turns the motor on when no call is arriving and turns it off for the exact `ring=1, vibrate_mode=1` case that should activate it. For one-bit signals, `!ring` and `~ring` have the same 0/1 result, so the failure is the reversed `ring` condition, not merely the choice of NOT operator.

Entry 85 - Q5b serial two's complementer, true one-hot Mealy FSM

The problem asks for one-hot encoding. The latest successful code uses `s0=2'b00` and `s1=2'b01`; `2'b00` has no asserted state bit, so it is not one-hot even though HDLBits accepts the external behavior. Functional tests cannot necessarily observe the internal encoding.

The older unsuccessful submission also omitted `zr=0` in one branch of its combinational block. That leaves `zr` unassigned on that path and infers a latch. Here is a one-hot implementation with every combinational output assigned:

```
localparam [1:0] BEFORE_FIRST_ONE = 2'b01;
localparam [1:0] AFTER_FIRST_ONE  = 2'b10;

reg [1:0] state, next_state;

always @(posedge clk or posedge areset) begin
    if (areset)
        state <= BEFORE_FIRST_ONE;
    else
        state <= next_state;
end

always @(*) begin
    next_state = state;
    z = 1'b0;

    case (1'b1)
        state[0]: begin
            z = x;                  // Copy zeros and the first one.
            if (x)
                next_state = AFTER_FIRST_ONE;
        end
        state[1]: begin
            z = ~x;                 // Invert all later bits.
            next_state = AFTER_FIRST_ONE;
        end
        default: begin
            next_state = BEFORE_FIRST_ONE;
            z = x;
        end
    endcase
end
```

The 3 September follow-up: entries 86-93

Kapil reported these eight problems completed in the current pass. The Chrome tab strip captured on 2026-09-03 contains the same eight HDLBits pages, in the same progress block. Because page-body attachment was unavailable during this follow-up, the evidence column records the confirmation without adding submission timestamps.

Entry	Problem	Attempt 2 state	Question/reference
86	Vectorr	Done	No questions asked
87	Dualedge	Done	Day 6 non-FSM Q&A PDF
88	Thermostat	Done	No questions asked
89	Sim/circuit5	Done	Day 6 non-FSM Q&A PDF
90	Exams/2014 q3fsm	Done	Standalone three-sample-window FSM PDF
91	Vector4	Done	No questions asked
92	Count15	Done	No questions asked
93	Popcount3	Done	No questions asked

The saved Edge conversations add three review topics:

- In Dualedge, `qpos | qneg` can preserve a stale 1 from the register that was not updated at the latest edge. The primary HDLBits solution selects `qpos` while `clk` is high and `qneg` while it is low. The expanded PDF now distinguishes simulator acceptance from FPGA implementability: ordinary fabric registers are

normally single-edge, whereas real DDR capture and launch use device-specific I/O resources such as AMD IDDR/ODDR or Intel/Altera DDIO. It also covers half-cycle timing, duty-cycle, glitch, metastability, reset, and XOR-startup concerns using primary vendor documentation.

- In Sim/circuit5, the waveform implies `c=0 -> b`, `c=1 -> e`, `c=2 -> a`, `c=3 -> d`, and all other selector values -> `4'hf`. Therefore `c` belongs in `case(c)`; input bus `b` is an identifier, not hexadecimal digit B.

- In Exams/2014 q3fsm, `ticks==2` means two samples have already been processed before the current third edge. Because nonblocking assignments expose old register values within the block, the third-sample decision uses `count + w`. The window counters must reset after every third sample, not only when the group contains exactly two ones. Both earlier wrong versions and the corrected two-state RTL are preserved in the standalone PDF.

Local verification

The reusable testbench day5_original_submission_selfcheck.sv checks:

- all 16 input combinations for entry 77;

- sampled 0-to-1 pulses for entry 74;

- overlapping `10101` recognition and immediate reset assertion for entry 75;

- LSB-first Moore and true one-hot Mealy two's-complement streams for entries 80 and 85;

- all 16 waveform input combinations for entry 81;

- 200 randomized concatenations for entry 82;

- sticky falling-edge capture, reset priority, and post-reset behavior for entry 83;

- all four Ringer truth-table rows and mutual exclusion for entry 84.

Icarus Verilog result:

```
PASS: Chrome-audit follow-up behavior checks passed (entries 74, 75, 77, 80-85).
```

The Day 6 checkers add:

```
PASS: all 8 possible q3fsm groups plus back-to-back grouping; saved reset-only-on-match
  bug diverged as expected
PASS: Dualedge MUX/XOR solutions and all 16 Circuit5 selector values; stale-register OR
  bug diverged as expected
```

Historical submission records

The table below records every original page in the audited range. For entries 1-85, dates are copied from the saved-submission selector displayed by HDLBits. For entries 86-93, `2026-09-03 current pass; user-confirmed` records Kapil's confirmation rather than a submission time.

No.	Original HDLBits page	Last success shown in Chrome	Last non-success shown in Chrome	Attempt 2 state	Question/reference
1	Lemmings1	8/29/2026, 3:33:16 PM	8/29/2026, 3:30:28 PM	Done	No explicit question recorded
2	Fsm serial	8/30/2026, 9:56:33 AM	8/30/2026, 9:55:55 AM	Done	Serial discussion PDF
3	Lemmings2	8/29/2026, 3:54:58 PM	8/29/2026, 3:51:51 PM	Done	No explicit question recorded
4	Fsm serialdata	8/30/2026, 10:24:16 AM	8/30/2026, 10:22:16 AM	Done	Serial discussion PDF
5	Lemmings3	8/29/2026, 4:21:22 PM	8/29/2026, 4:21:03 PM	Done	No explicit question recorded
6	Fsm serialdp	8/30/2026, 10:55:13 AM	8/30/2026, 10:53:43 AM	Done	Serial discussion PDF
7	Lemmings4	8/29/2026, 9:14:52 PM	8/29/2026, 9:12:30 PM	Done	Combined questions PDF
8	Fsm hdlc	8/29/2026, 11:20:34 PM	8/29/2026, 11:14:37 PM	Done	HDLC discussion PDF
9	Conditional	8/30/2026, 3:40:27 PM	none	Done	No explicit question recorded
10	Dff	8/30/2026, 3:42:19 PM	none	Done	No explicit question recorded
11	Step one	8/30/2026, 3:42:48 PM	none	Done	No explicit question recorded
12	Fsm1	8/30/2026, 3:49:25 PM	8/30/2026, 3:48:09 PM	Done	No explicit question recorded
13	Bugs mux2	8/30/2026, 4:09:04 PM	8/30/2026, 4:08:09 PM	Done	Combined questions PDF
14	Reduction	8/30/2026, 4:10:13 PM	6/24/2026, 6:14:19 PM	Done	No explicit question recorded
15	Dff8	8/30/2026, 4:12:23 PM	none	Done	No explicit question recorded
16	Zero	8/30/2026, 4:12:48 PM	none	Done	No explicit question recorded
17	Fsm1s	8/30/2026, 4:21:45 PM	8/30/2026, 4:23:11 PM	Done	No explicit question recorded
18	Gates100	8/30/2026, 4:14:01 PM	none	Done	No explicit question recorded
19	Dff8r	8/30/2026, 5:06:27 PM	6/25/2026, 10:23:54 PM	Done	Combined questions PDF
20	Wire	8/30/2026, 5:06:45 PM	none	Done	Combined questions PDF
21	Bugs nand3	8/30/2026, 5:12:57 PM	8/30/2026, 5:07:56 PM	Done	Combined questions PDF
22	Fsm2	8/30/2026, 5:15:53 PM	8/30/2026, 5:15:28 PM	Done	Combined questions PDF
23	Vector100r	8/30/2026, 5:21:27 PM	6/24/2026, 6:21:32 PM	Done	Combined questions PDF
24	Dff8p	8/30/2026, 6:12:50 PM	8/30/2026, 5:30:37 PM	Done	Combined questions PDF
25	Wire4	8/30/2026, 5:31:49 PM	6/23/2026, 5:49:49 PM	Done	Combined questions PDF
26	Popcount255	8/30/2026, 5:50:16 PM	8/30/2026, 5:46:15 PM	Done	Combined questions PDF
27	Fsm2s	8/30/2026, 5:54:41 PM	8/30/2026, 5:53:48 PM	Done	Combined questions PDF
28	Dff8ar	8/30/2026, 5:58:16 PM	6/25/2026, 10:28:54 PM	Done	Combined questions PDF
29	Bugs mux4	8/30/2026, 6:09:06 PM	8/30/2026, 6:08:27 PM	Done	Combined questions PDF
30	Adder100i	8/30/2026, 6:29:42 PM	8/30/2026, 6:29:17 PM	Done	Combined questions PDF

No.	Original HDLBits page	Last success shown in Chrome	Last non-success shown in Chrome	Attempt 2 state	Question/reference
31	Notgate	8/30/2026, 7:09:37 PM	none	Done	Combined questions PDF
32	Fsm3comb	8/30/2026, 11:45:51 PM	8/30/2026, 11:43:15 PM	Done	No explicit question recorded
33	Dff16e	8/30/2026, 7:26:48 PM	6/25/2026, 10:39:59 PM	Done	No explicit question recorded
34	Bcdadd100	8/30/2026, 9:57:17 PM	8/30/2026, 9:54:23 PM	Done	No explicit question recorded
35	Andgate	8/30/2026, 7:11:50 PM	none	Done	No explicit question recorded
36	Bugs addsubz	8/30/2026, 10:07:09 PM	8/30/2026, 10:02:03 PM	Done	Combined questions PDF
37	Exams/m2014 q4h	8/30/2026, 7:21:33 PM	none	Done	No explicit question recorded
38	Exams/m2014 q4a	8/30/2026, 7:32:02 PM	8/30/2026, 7:30:55 PM	Done	Combined questions PDF
39	Fsm3onehot	8/30/2026, 10:36:07 PM	8/30/2026, 10:35:24 PM	Done	No explicit question recorded
40	Norgate	8/30/2026, 7:22:24 PM	6/24/2026, 12:31:16 AM	Done	No explicit question recorded
41	Exams/m2014 q4i	8/30/2026, 7:22:37 PM	none	Done	No explicit question recorded
42	Exams/m2014 q4b	8/30/2026, 7:37:09 PM	none	Done	No explicit question recorded
43	Fsm3	8/30/2026, 10:57:04 PM	none	Done	No explicit question recorded
44	Xnorgate	8/30/2026, 7:38:46 PM	8/30/2026, 7:38:25 PM	Done	No explicit question recorded
45	Exams/m2014 q4e	8/30/2026, 7:33:30 PM	6/24/2026, 11:32:45 PM	Done	No explicit question recorded
46	Exams/m2014 q4c	8/30/2026, 10:40:45 PM	none	Done	No explicit question recorded
47	Always case	8/30/2026, 10:43:52 PM	8/30/2026, 10:43:30 PM	Done	No explicit question recorded
48	Fsm3s	8/30/2026, 10:54:27 PM	8/30/2026, 10:52:35 PM	Done	No explicit question recorded
49	Wire decl	8/30/2026, 10:46:25 PM	none	Done	No explicit question recorded
50	Exams/m2014 q4f	8/30/2026, 7:35:50 PM	8/30/2026, 7:35:13 PM	Done	No explicit question recorded
51	Exams/m2014 q4d	8/31/2026, 9:32:17 AM	8/31/2026, 9:31:28 AM	Done	No explicit question recorded
52	Exams/m2014 q4g	8/31/2026, 9:34:10 AM	none	Done	No explicit question recorded
53	7458	8/31/2026, 9:39:14 AM	none	Done	No explicit question recorded
54	Exams/ece241 2013 q4	9/1/2026, 12:21:28 AM	9/1/2026, 12:16:46 AM	Done	Recovered questions PDF
55	Sim/circuit1	8/31/2026, 8:36:12 PM	6/28/2026, 1:32:14 AM	Done	No explicit question recorded
56	Mt2015 muxdff	9/1/2026, 12:32:44 AM	9/1/2026, 12:32:23 AM	Done	No explicit question recorded
57	Gates	9/1/2026, 12:39:20 AM	9/1/2026, 12:38:40 AM	Done	No explicit question recorded
58	Vector0	9/1/2026, 12:43:49 AM	9/1/2026, 12:40:57 AM	Done	No explicit question recorded
59	Fsm onehot	9/1/2026, 12:58:13 AM	7/5/2026, 5:07:43 PM	Done	No explicit question recorded
60	Exams/2014 q4a	9/1/2026, 1:04:13 AM	6/26/2026, 12:19:52 AM	Done	No explicit question recorded
61	7420	9/1/2026, 1:28:08 AM	none	Done	No explicit question recorded
62	Vector1	9/1/2026, 1:30:34 AM	6/23/2026, 9:44:41 PM	Done	No explicit question recorded
63	Sim/circuit2	9/1/2026, 11:00:02 AM	9/1/2026, 10:53:57 AM	Done	No explicit question recorded

No.	Original HDLBits page	Last success shown in Chrome	Last non-success shown in Chrome	Attempt 2 state	Question/reference
64	Fsm ps2	9/1/2026, 4:25:35 PM	9/1/2026, 4:26:58 PM	Done	No explicit question recorded
65	Truthtable1	9/1/2026, 5:12:37 PM	none	Done	No explicit question recorded
66	Exams/ece241 2014 q4	9/1/2026, 5:28:19 PM	9/1/2026, 5:26:37 PM	Done	Day 4 questions PDF
67	Vector2	9/1/2026, 5:40:40 PM	9/1/2026, 5:38:14 PM	Done	Day 4 questions PDF
68	Mt2015 eq2	9/1/2026, 6:30:47 PM	none	Done	No explicit question recorded
69	Exams/ece241 2013 q7	9/1/2026, 6:29:32 PM	none	Done	No explicit question recorded
70	Fsm ps2data	9/1/2026, 4:46:09 PM	9/1/2026, 4:43:36 PM	Done	Day 4 questions PDF
71	Sim/circuit3	9/1/2026, 10:21:36 PM	9/1/2026, 10:17:52 PM	Done	No explicit question recorded
72	Mt2015 q4a	9/1/2026, 10:22:14 PM	none	Done	No explicit question recorded
73	Vectorgates	9/1/2026, 10:24:57 PM	9/1/2026, 10:23:57 PM	Done	No explicit question recorded
74	Edgedetect	9/1/2026, 10:28:38 PM	9/1/2026, 9:58:30 PM	Done	Recovered questions PDF
75	Exams/ece241 2013 q8	9/1/2026, 10:37:57 PM	9/1/2026, 10:36:46 PM	Done	Recovered questions PDF
76	Mt2015 q4b	9/1/2026, 10:38:41 PM	none	Done	No explicit question recorded
77	Gates4	9/1/2026, 10:44:37 PM	6/23/2026, 10:15:40 PM	Done	Recovered questions PDF
78	Edgedetect2	9/1/2026, 10:47:42 PM	9/1/2026, 10:47:07 PM	Done	No explicit question recorded
79	Mt2015 q4	9/1/2026, 11:01:04 PM	9/1/2026, 11:00:16 PM	Done	No explicit question recorded
80	Exams/ece241 2014 q5a	9/2/2026, 4:28:11 PM	9/2/2026, 4:18:12 PM	Done	Entry 80 Moore FSM PDF
81	Sim/circuit4	9/1/2026, 11:26:08 PM	6/28/2026, 3:34:12 PM	Done	Day 5 submission review
82	Vector3	9/1/2026, 11:23:20 PM	9/1/2026, 11:19:13 PM	Done	Day 5 submission review
83	Edgecapture	9/2/2026, 5:00:08 PM	9/2/2026, 4:57:13 PM	Done	Day 5 submission review
84	Ringer	9/2/2026, 4:54:21 PM	9/2/2026, 4:53:47 PM	Done	Day 5 submission review
85	Exams/ece241 2014 q5b	9/2/2026, 4:44:54 PM	9/2/2026, 4:31:02 PM	Done	Day 5 submission review
86	Vectorr	2026-09-03 current pass; user-confirmed	not captured in follow-up	Done	No explicit question recorded
87	Dualedge	2026-09-03 current pass; user-confirmed	not captured in follow-up	Done	Day 6 non-FSM Q&A PDF
88	Thermostat	2026-09-03 current pass; user-confirmed	not captured in follow-up	Done	No explicit question recorded
89	Sim/circuit5	2026-09-03 current pass; user-confirmed	not captured in follow-up	Done	Day 6 non-FSM Q&A PDF
90	Exams/2014 q3fsm	2026-09-03 current pass; user-confirmed	not captured in follow-up	Done	Standalone three-sample-window FSM PDF
91	Vector4	2026-09-03 current pass; user-confirmed	not captured in follow-up	Done	No explicit question recorded

No.	Original HDLBits page	Last success shown in Chrome	Last non-success shown in Chrome	Attempt 2 state	Question/reference
92	Count15	2026-09-03 current pass; user-confirmed	not captured in follow-up	Done	No explicit question recorded
93	Popcount3	2026-09-03 current pass; user-confirmed	not captured in follow-up	Done	No explicit question recorded

At the 3 September checkpoint, entries 1-93 were Done and entries 94-178 had not yet been included in the review. The earlier 7 September document-review checkpoint recorded 151 Done and 27 Pending. These are historical counts; use the tracker for current progress.